

SOFTWARE DEVELOPER EDITION

Bash Field Guide for Developers

A concise, visual reference for daily terminal work

Navigate, inspect, search, automate, diagnose and recover - safely.

```
$ pwd

/home/dev/projects/launch-ready-app

$ rg "TODO" src | wc -l

18

$ npm test && npm run build

Tests passed. Build completed in 8.4s

$ _
```

NAVIGATE

pwd cd ls

SEARCH

rg grep find

COMPOSE

| > && \$(...)

RECOVER

history Ctrl+R

SAFETY RULE Preview destructive operations, verify the path, then execute.

Interactive contents

Click any section title or page number to jump directly to that topic. Every content page includes a return link in the footer.

Filesystem Navigation	3
Files and Directories	4
Viewing File Content	5
Search and Text Processing	6
Permissions and Environment	8
Pipes, Redirection and Logic	9
Processes, Ports and Jobs	10
Networking and HTTP	11
Archives and Compression	12
Git and Dependency Workflows	13
Disk and System Diagnostics	15
High-Value One-Liners	16
Scripts and Keyboard Shortcuts	18
Safety and Daily Command Map	19

COMMAND ANATOMY

command [options/flags] [input/path]

Example: `grep -rn "TODO" src` means run **grep**, recurse with **-r**, show line numbers with **-n**, search for **TODO**, and start in **src**.

SAFE OPERATING LOOP

Preview the targets -> **verify** the path and flags -> **execute** -> **inspect** the result. This matters most for `rm`, `mv`, `chmod`, `chown`, `git clean` and commands using `sudo`.

Filesystem Navigation

Know where you are, move deliberately and inspect directory structure.



TASK	COMMAND	WHEN / WHY
Show current path	<code>pwd</code>	Print the absolute path of the working directory.
Enter a directory	<code>cd project</code>	Move into a child directory.
Go up one level	<code>cd ..</code>	Move to the parent directory.
Go home	<code>cd ~</code> or: <code>cd</code>	Jump to the current user's home directory.
Return to previous	<code>cd -</code>	Toggle between the current and previous directories.
Basic listing	<code>ls</code>	List visible entries in the current directory.
Include hidden	<code>ls -a</code>	Show dotfiles such as <code>.git</code> , <code>.env</code> and <code>.vscode</code> .
Detailed listing	<code>ls -lh</code>	Show permissions, owner, size and time in readable units.
Sort by time / size	<code>ls -lt</code> <code>ls -lhS</code>	Newest-first or largest-first views.
Recursive listing	<code>ls -R</code>	Walk through nested directories; avoid on very large trees.

FAST CONTEXT RESET

Use `pwd`, then `ls -la`. If you entered the wrong directory, `cd -` returns to the previous location.

02 / DEVELOPER SHELL GUIDE

Files and Directories

Create, copy, move, rename and remove filesystem objects.



TASK	COMMAND	WHEN / WHY
Create empty file	<code>touch README.md</code>	Create the file or update its modification time.
Create directory	<code>mkdir components</code>	Create one directory.
Create nested path	<code>mkdir -p src/components/Button</code>	Create missing parent directories automatically.
Create several	<code>touch a.js b.js mkdir api tests</code>	Create multiple files or directories in one command.
Copy file	<code>cp .env.example .env</code>	Duplicate a file under a new name or path.
Copy directory	<code>cp -r src src-backup</code>	Recursively copy the directory tree.
Move file	<code>mv app.js src/app.js</code>	Move a file to another directory.
Rename	<code>mv old-name.txt new-name.txt</code>	Rename files and directories with mv.
Delete file	<code>rm file.txt</code>	Remove a file permanently.
Confirm deletion	<code>rm -i file.txt</code>	Prompt before deleting; useful while learning.
Delete empty dir	<code>rmdir empty-folder</code>	Only succeeds when the directory is empty.
Delete directory	<code>rm -r folder</code>	Recursively remove the directory and contents.

DELETION HAS NO UNDO

Prefer `rm -i` while learning. Before any recursive delete, run `pwd` and list the exact target.

Viewing File Content

Read short files, page through large files and follow live logs.



TASK	COMMAND	WHEN / WHY
Print whole file	<code>cat package.json</code>	Best for short files; avoid flooding the terminal with large files.
Paged viewing	<code>less app.log</code>	Scroll, search and quit without loading everything visually at once.
First lines	<code>head -n 20 file.txt</code>	Inspect headers or the beginning of data.
Last lines	<code>tail -n 50 file.txt</code>	Inspect recent entries in logs or generated output.
Follow live logs	<code>tail -f logs/server.log</code>	Stream new lines until Ctrl+C.
Add line numbers	<code>nl file.txt</code>	Display numbered lines without modifying the source.
Number via cat	<code>cat -n file.txt</code>	Alternative line-number view.
Clear terminal	<code>clear</code> # Ctrl+L	Reset the visible screen without deleting history.

INSIDE LESS

Space: next page b: previous page /word: search n: next match q: quit

LOG WORKFLOW

Use `tail -n 100 app.log` for context, then `tail -f app.log` to watch new events. Stop with Ctrl+C.

WRITE OR EDIT FILE CONTENT

Redirection changes where command output goes; use `>` to replace and `>>` to append.



GOAL	COMMAND	WHAT IT DOES
Append one line	<code>echo "New content" >> file.txt</code>	Adds text at the end and keeps existing content.
Replace whole file	<code>echo "New content" > file.txt</code>	Overwrites the file. Double-check the path first.
Append typed lines	<code>cat >> file.txt</code>	Type lines, then press Ctrl+D to save and exit.
Replace with typed lines	<code>cat > file.txt</code>	Replaces existing content; Ctrl+D finishes input.
Append a text block	<code>cat << 'EOF' >> file.txt</code>	Appends every line until the closing EOF marker.
Edit manually	<code>nano file.txt</code>	Save: Ctrl+O, Enter. Exit: Ctrl+X.

REDIRECTION RULE

`>` replaces the file; `>>` appends to it. A mistaken `>` can erase existing content immediately.

WHY CAT CAN WRITE

cat reads your keyboard input. The shell redirects that output into the file. Press Ctrl+D to send end-of-input.

MULTI-LINE APPEND EXAMPLE

```
cat << 'EOF' >> file.txt
First new line
Second new line
EOF
```

04 / DEVELOPER SHELL GUIDE

Search and Text Processing

Find files, locate text and transform line-oriented data.



Find files and search source code

TASK	COMMAND	WHEN / WHY
Find exact name	<code>find . -name "app.js"</code>	Search from the current directory by filename.
Find by extension	<code>find . -type f -name "*.js"</code>	Limit results to files matching a pattern.
Find directories	<code>find . -type d -name "node_modules"</code>	Search only directories.
Find large files	<code>find . -type f -size +100M</code>	Locate files larger than 100 MB.
Find recent files	<code>find . -type f -mtime -1</code>	Files modified during the last 24 hours.
Find last 10 min	<code>find . -type f -mmin -10</code>	Useful during build or runtime debugging.
Search one file	<code>grep "hello" file.txt</code>	Print lines containing the exact text.
Ignore case	<code>grep -i "error" app.log</code>	Case-insensitive matching.
Recursive + lines	<code>grep -rn "TODO" src</code>	Search a tree and show file paths with line numbers.
Exclude directory	<code>grep -rn "TODO" . --exclude-dir=node_modules</code>	Skip large or irrelevant directories.
Filter file type	<code>grep -rn "import" --include="*.js" src</code>	Search only selected filename patterns.
Faster alternative	<code>rg "useEffect" -g "*.tsx"</code>	ripgrep is usually faster and respects .gitignore by default.

DEFAULT SEARCH CHOICE

Use `rg` for fast source-code search when installed. Use `grep` when portability matters and `find` when the target is a file or directory rather than its contents.

Transform, count and aggregate text

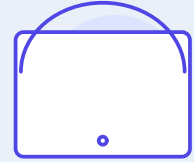
TASK	COMMAND	WHEN / WHY
Count lines	<code>wc -l file.txt</code>	Count newline-terminated lines.
Sort	<code>sort names.txt</code>	Sort text lexicographically.
Unique values	<code>sort names.txt uniq</code>	uniq removes adjacent duplicates, so sort first.
Count uniques	<code>sort names.txt uniq -c</code>	Count how often each unique line appears.
Only duplicates	<code>sort names.txt uniq -d</code>	Print values occurring more than once.
Top frequencies	<code>sort file.txt uniq -c sort -nr head</code>	Rank repeated lines by count.
Remove blank lines	<code>grep -v '^[[:space:]]*\$' file.txt</code>	Remove empty and whitespace-only lines.
Replace on Linux	<code>sed -i 's/old/new/g' file.txt</code>	Edit the file in place.
Replace on macOS	<code>sed -i '' 's/old/new/g' file.txt</code>	BSD sed requires an empty backup suffix.
First column	<code>awk '{print \$1}' access.log</code>	Extract a whitespace-delimited field.
Safe file passing	<code>find . -name '*.js' -print0 xargs -0 wc -l</code>	Handles spaces and special characters in filenames.

PIPELINE PRINCIPLE

Most Unix text tools do one focused job. Compose them with pipes instead of searching for one oversized command.

Permissions and Environment

Control access, executable files and process configuration.



TASK	COMMAND	WHEN / WHY
Inspect permissions	<code>ls -l</code>	Read the owner/group/other permission triplets.
Make executable	<code>chmod +x script.sh</code>	Allow the script to run as <code>./script.sh</code> .
Typical script mode	<code>chmod 755 script.sh</code>	Owner: <code>rx</code> ; group and others: <code>r-x</code> .
Typical file mode	<code>chmod 644 file.txt</code>	Owner: <code>rw</code> ; group and others: <code>read-only</code> .
Change ownership	<code>sudo chown user:group file.txt</code>	Use <code>sudo</code> only when ownership requires elevated privileges.
List variables	<code>env</code>	Print environment variables available to the process.
Read a variable	<code>echo \$PATH</code>	Expand and print one variable.
Temporary variable	<code>export PORT=3000</code>	Available to child processes in the current shell session.
One-command variable	<code>PORT=3000 npm start</code>	Set a value only for the launched command.
Persist in Bash	<code>echo 'export NODE_ENV=development' >> ~/.bashrc</code>	Append configuration, then run <code>source ~/.bashrc</code> .
Persist in Zsh	<code>echo 'export NODE_ENV=development' >> ~/.zshrc</code>	Append configuration, then run <code>source ~/.zshrc</code> .

PERMISSION NUMBERS

7 = `rx`, 6 = `rw-`, 5 = `r-x`, 4 = `r--`. A mode such as 755 applies digits to owner, group and others.

SECRETS

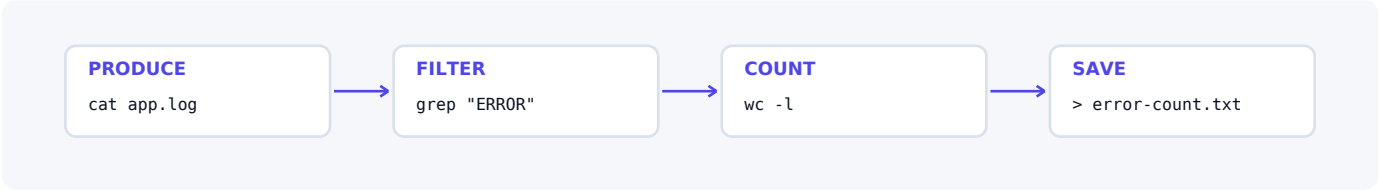
Viewing `.env` is convenient, but never commit API keys, passwords or tokens. Keep secret files excluded with `.gitignore` and use a secrets manager in production.

Pipes, Redirection and Logic

Connect commands, route output and control execution order.



TASK	COMMAND	WHEN / WHY
Overwrite file	<code>echo "Hello" > hello.txt</code>	Replace the destination file with command output.
Append to file	<code>echo "More" >> hello.txt</code>	Add output at the end without replacing existing content.
Errors only	<code>npm start 2> error.log</code>	Redirect standard error to a file.
Output + errors	<code>npm start > output.log 2>&1</code>	Send stdout and stderr to the same destination.
Pipe output	<code>grep "ERROR" app.log wc -l</code>	Use one command's output as another command's input.
Run on success	<code>npm install && npm test</code>	Execute the next command only when the previous one succeeds.
Run on failure	<code>npm test echo "Tests failed"</code>	Execute the fallback only when the first command fails.
Always continue	<code>npm install; npm build; npm test</code>	Run in sequence regardless of each exit status.
Command substitution	<code>kill \$(lsof -ti :3000)</code>	Run an inner command and insert its output.
Background process	<code>npm run dev &</code>	Start a command as a shell job in the background.



READ LEFT TO RIGHT

In `cat app.log | grep "ERROR" | wc -l > count.txt`, each stage receives the previous output. Prefer `grep "ERROR" app.log` when a command can read the file directly.

Processes, Ports and Jobs

Identify running programs, free ports and manage foreground/background work.



TASK	COMMAND	WHEN / WHY
Current shell processes	<code>ps</code>	Show processes attached to the current terminal.
All processes	<code>ps aux</code>	Detailed system-wide process snapshot.
Search process	<code>ps aux grep node</code>	Locate processes by name; the grep process may also appear.
Graceful stop	<code>kill 12345</code>	Send SIGTERM so the process can clean up.
Force stop	<code>kill -9 12345</code>	Send SIGKILL only when normal termination fails.
Find port owner	<code>lsof -i :3000</code>	Show the process using TCP/UDP port 3000.
PID on port	<code>lsof -ti :3000</code>	Return only the matching process ID.
Kill port process	<code>kill \$(lsof -ti :3000)</code>	Prefer normal kill before using -9.
Live process view	<code>top #</code> or <code>htop</code>	Monitor CPU, memory and running processes.
Shell jobs	<code>jobs fg</code>	List background jobs or bring one to the foreground.

TERMINATE PROGRESSIVELY

Try `kill PID` first. Use `kill -9 PID` only when the process ignores normal termination, because SIGKILL prevents cleanup.

08 / DEVELOPER SHELL GUIDE

Networking and HTTP

Check connectivity, inspect endpoints and make API requests.



TASK	COMMAND	WHEN / WHY
Connectivity	<code>ping example.com</code>	Send ICMP probes; stop with Ctrl+C.
HTTP GET	<code>curl https://example.com</code>	Print the response body.
Headers only	<code>curl -I https://example.com</code>	Inspect status and response headers.
Follow redirects	<code>curl -L https://example.com</code>	Follow 3xx redirect responses.
Download by name	<code>curl -O https://example.com/file.zip</code>	Save using the remote filename.
JSON POST	<code>curl -X POST URL -H 'Content-Type: application/json' -d '{...}'</code>	Send a JSON request body.
Website status	<code>curl -Is https://example.com head -n 1</code>	Quickly print the first HTTP status line.
DNS lookup	<code>nslookup example.com</code>	Resolve host records with a widely available utility.
Detailed DNS	<code>dig example.com</code>	Inspect DNS answers, authority and timing information.

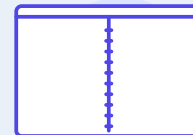
JSON POST EXAMPLE

```
curl -X POST https://api.example.com/users \
  -H 'Content-Type: application/json' \
  -d '{"name": "Satyam", "role": "developer"}'
```

09 / DEVELOPER SHELL GUIDE

Archives and Compression

Package projects, extract downloads and create timestamped backups.



TASK	COMMAND	WHEN / WHY
Create ZIP	<code>zip -r project.zip project-folder</code>	Recursively compress a directory into ZIP.
Extract ZIP	<code>unzip project.zip</code>	Extract to the current directory.
Create tar.gz	<code>tar -czvf project.tar.gz project-folder</code>	Create (-c), gzip (-z), verbose (-v), filename (-f).
Extract tar.gz	<code>tar -xzvf project.tar.gz</code>	Extract (-x) a gzip-compressed archive.
Extract elsewhere	<code>tar -xzvf project.tar.gz -C destination</code>	Choose a destination directory.
Timestamped backup	<code>tar -czvf "backup-\$(date +%F-%H-%M-%S).tar.gz" project</code>	Embed the current date and time in the filename.

TAR FLAG MEMORY

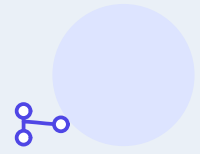
Create: -c; extract: -x; gzip: -z; verbose: -v; archive filename follows -f.

VERIFY BEFORE RESTORING

List an archive first with `tar -tzf backup.tar.gz` or `unzip -l project.zip` when you do not trust its directory layout.

Git and Dependency Workflows

Use the shell to inspect repositories and run repeatable project setup.



Git from the terminal

TASK	COMMAND	WHEN / WHY
Repository status	<code>git status</code>	Show branch state, staged changes and untracked files.
Initialize	<code>git init</code>	Create a new local Git repository.
Current branches	<code>git branch</code>	List local branches and mark the current one.
Remote URLs	<code>git remote -v</code>	Verify fetch and push remotes.
Stage changes	<code>git add .</code>	Stage changes below the current directory; review first.
Commit	<code>git commit -m "Meaningful message"</code>	Create a commit from staged changes.
Inspect changes	<code>git diff</code>	Review unstaged line-level changes.
Preview cleanup	<code>git clean -fdn</code>	Dry-run removal of untracked files and directories.
Run cleanup	<code>git clean -fd</code>	Permanently remove previewed untracked content.

REVIEW BEFORE MUTATION

Run `git status` and `git diff` before staging or cleaning. For cleanup, the `-n` dry run is mandatory.

Node.js / npm

TASK	COMMAND	WHEN / WHY
Install project deps	<code>npm install # or npm i</code>	Install dependencies from package.json.
Add dependency	<code>npm install express</code>	Add a runtime dependency.
Add dev dependency	<code>npm install -D nodemon</code>	Add tooling used during development.
Run script	<code>npm run dev</code>	Execute a script declared in package.json.
List scripts	<code>npm run</code>	Show available package scripts.
Clean reinstall	<code>rm -rf node_modules && npm install</code>	Rebuild installed dependencies; preserve lockfile unless intentionally regenerating.

Python virtual environments

TASK	COMMAND	WHEN / WHY
Create virtual env	<code>python3 -m venv .venv</code>	Create an isolated Python environment.
Activate macOS/Linux	<code>source .venv/bin/activate</code>	Activate the environment in the current shell.
Activate Git Bash	<code>source .venv/Scripts/activate</code>	Windows Git Bash activation path.
Install requirements	<code>pip install -r requirements.txt</code>	Install pinned project dependencies.
Freeze requirements	<code>pip freeze > requirements.txt</code>	Capture installed package versions.

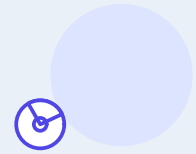
LOCKFILES ARE EVIDENCE

Deleting `package-lock.json` can change dependency resolution. Remove it only when you intentionally want to regenerate the dependency graph.

11 / DEVELOPER SHELL GUIDE

Disk and System Diagnostics

Understand storage, memory, tool locations and execution time.



TASK	COMMAND	WHEN / WHY
Filesystem usage	<code>df -h</code>	Show mounted filesystem capacity and free space.
Directory size	<code>du -sh node_modules</code>	Summarize one directory's disk use.
Child sizes	<code>du -sh *</code>	Compare immediate entries in the current directory.
Sort by size	<code>du -sh * sort -h</code>	Order entries by human-readable size.
Memory usage	<code>free -h</code>	Linux memory and swap summary.
OS and kernel	<code>uname -a</code>	Print operating-system and kernel information.
Command location	<code>which node</code>	Show the executable resolved from PATH.
Tool versions	<code>node -v; npm -v; git --version; python3 --version</code>	Capture runtime versions during troubleshooting.
Measure duration	<code>time npm run build</code>	Report real, user and system execution time.

A PRACTICAL DIAGNOSIS ORDER

Check `df -h` for filesystem capacity, `du -sh *` for local offenders, then process and memory tools if the issue is runtime-related.

12 / DEVELOPER SHELL GUIDE

High-Value One-Liners

Reusable combinations for codebases, logs, storage and automation.



Codebase and filesystem recipes

TASK	COMMAND	WHEN / WHY
Count visible entries	<code>ls wc -l</code>	Counts visible names printed by ls, not hidden entries.
Count files recursively	<code>find . -type f wc -l</code>	Count all regular files under the current directory.
Count JS lines	<code>find . -name '*.js' -type f -print0 xargs -0 wc -l</code>	Count lines while safely handling filename spaces.
Largest files	<code>find . -type f -exec du -h {} + sort -rh head -n 10</code>	Show the ten largest files.
Find TODOs	<code>grep -rn 'TODO' . --exclude-dir=node_modules</code>	Locate TODO comments outside dependencies.
Remove .DS_Store	<code>find . -name '.DS_Store' -type f -print</code>	Preview matches first; replace -print with -delete after review.
Large files >50 MB	<code>find . -type f -size +50M -exec ls -lh {} \;</code>	Display metadata for large files.
Live error stream	<code>tail -f app.log grep --line-buffered 'ERROR'</code>	Follow only new matching log entries.

PREVIEW-TO-DELETE PATTERN

Start with -print. Only after confirming the exact matches should you replace it with -delete or pass the results to rm.

Logs, repositories and workflow recipes

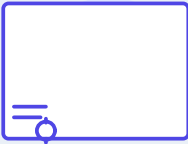
TASK	COMMAND	WHEN / WHY
Top client IPs	<code>awk '{print \$1}' access.log sort uniq -c sort -nr head</code>	Count and rank the first log field.
Failed HTTP requests	<code>awk '\$9 >= 400 {print}' access.log</code>	Print rows whose ninth field is an HTTP error status.
Scan rough secret names	<code>grep -rE 'API_KEY SECRET TOKEN PASSWORD' . --exclude-dir=.git</code>	Heuristic check; do not treat as a complete secret scanner.
Format every JS file	<code>find . -name '*.js' -type f -print0 xargs -0 prettier --write</code>	Run Prettier safely on matching files.
Branch in each repo	<code>find . -maxdepth 3 -type d -name .git while read d; do r=\$(dirname "\$d"); git -C "\$r" branch --show-current; done</code>	Inspect nested repositories.
Preview logs to delete	<code>find . -type f -name '*.log' -print</code>	Review exact matches before deletion.
Delete reviewed logs	<code>find . -type f -name '*.log' -delete</code>	Permanent deletion; run only after a preview.
Check build sequence	<code>npm install && npm run build && npm test</code>	Stop automatically at the first failed step.

QUOTE PATHS AND VARIABLES

Inside loops and command substitutions, quote variables such as "\$r" to preserve spaces and prevent unintended word splitting.

Scripts and Keyboard Shortcuts

Turn repeated commands into safe scripts and work faster at the prompt.



A minimal fail-fast Bash script

```
SETUP.SH
#!/usr/bin/env bash
set -euo pipefail

echo 'Installing dependencies...'
npm install

echo 'Running tests...'
npm test

echo 'Done.'
```

RUN THE SCRIPT
Save as setup.sh, run `chmod +x setup.sh`, then execute `./setup.sh`. `set -euo pipefail` stops on failures, unset variables and failed pipeline stages.

High-impact terminal shortcuts

TASK	COMMAND	WHEN / WHY
Stop current command	Ctrl+C	Interrupt the foreground process.
Clear screen	Ctrl+L	Same visible result as clear.
Line start / end	Ctrl+A / Ctrl+E	Move the cursor to the beginning or end.
Delete before / after	Ctrl+U / Ctrl+K	Remove text before or after the cursor.
Delete previous word	Ctrl+W	Efficiently edit a command argument.
Search history	Ctrl+R	Incrementally find previously run commands.
History navigation	Up / Down	Move through previous and next commands.
Autocomplete	Tab	Complete paths and commands; press twice to list options.
Repeat previous	!!	Run the immediately previous command again.
Repeat by prefix	!npm	Run the latest history entry beginning with npm.

Safety and Daily Command Map

Respect destructive operations and build a small, reliable daily toolkit.



Commands that deserve a deliberate pause

TASK	COMMAND	WHEN / WHY
Recursive force delete	<code>rm -rf path</code>	No confirmation and no recycle bin. Verify the expanded path first.
Root deletion	<code>rm -rf /</code>	Catastrophic. Modern systems may block it, but never experiment with it.
Wildcard deletion	<code>rm -rf *</code>	Deletes current-directory contents; hidden files may behave differently.
Overwrite output	<code>> important.txt</code>	Truncates the destination before writing. Use <code>>></code> to append.
World-writable	<code>chmod -R 777 .</code>	Grants broad permissions and often creates security problems.
Recursive ownership	<code>sudo chown -R user:group /path</code>	A wrong path can damage system ownership.
Git cleanup	<code>git clean -fd</code>	Removes untracked files permanently; always run <code>git clean -fdn</code> first.
Force kill	<code>kill -9 PID</code>	Prevents cleanup and may leave corrupted state; use normal kill first.

THREE-SECOND SAFETY CHECK

Before pressing Enter: **1)** read the full expanded command, **2)** verify the working directory, **3)** confirm whether the action is reversible.

Daily command decision map

Start with the task, not with a command name.

Where am I?
pwd | ls -la

Inspect content
less | head | tail -f

Diagnose runtime
ps aux | lsof | top

Change safely
preview -> confirm -> execute

Move or create
cd | mkdir -p | touch

Find something
rg | grep -rn | find

Combine results
| > >> && || \$(...)

Recover context
history | Ctrl+R | cd -

Commands worth learning first

DAILY CORE

pwd	ls -la	cd folder	cd ..	mkdir -p path
touch file	cp source dest	mv old new	rm -i file	
less file	head -n 20 file	tail -f app.log		
rg 'text'	grep -rn 'text' .	find . -name '*.js'		
chmod +x script.sh	ps aux	lsof -i :3000	kill PID	
curl -I https://example.com		tar -czvf backup.tar.gz folder		
git status	git diff	history	Ctrl+R	

THE REAL BASH SKILL

Do not memorize every flag. Learn how commands accept input, produce output and expose exit status. Then combine them using pipes |, redirection > >>, conditions && || and substitution \$(...).

End of field guide

Use the Contents link in the footer to jump back to any section.